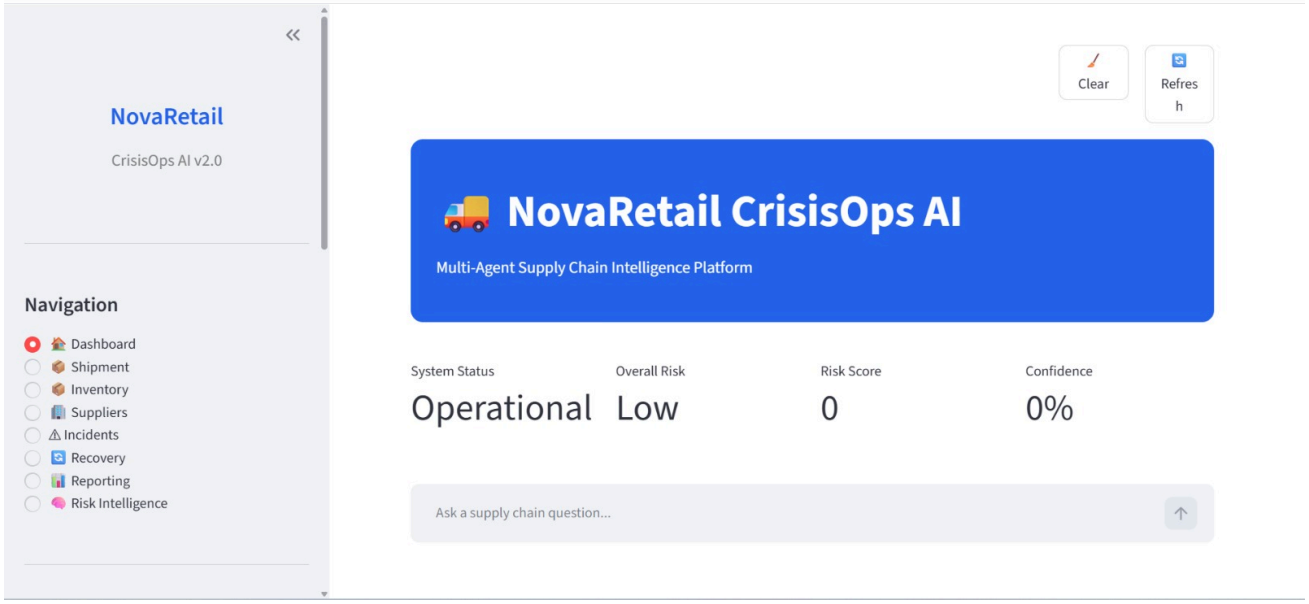
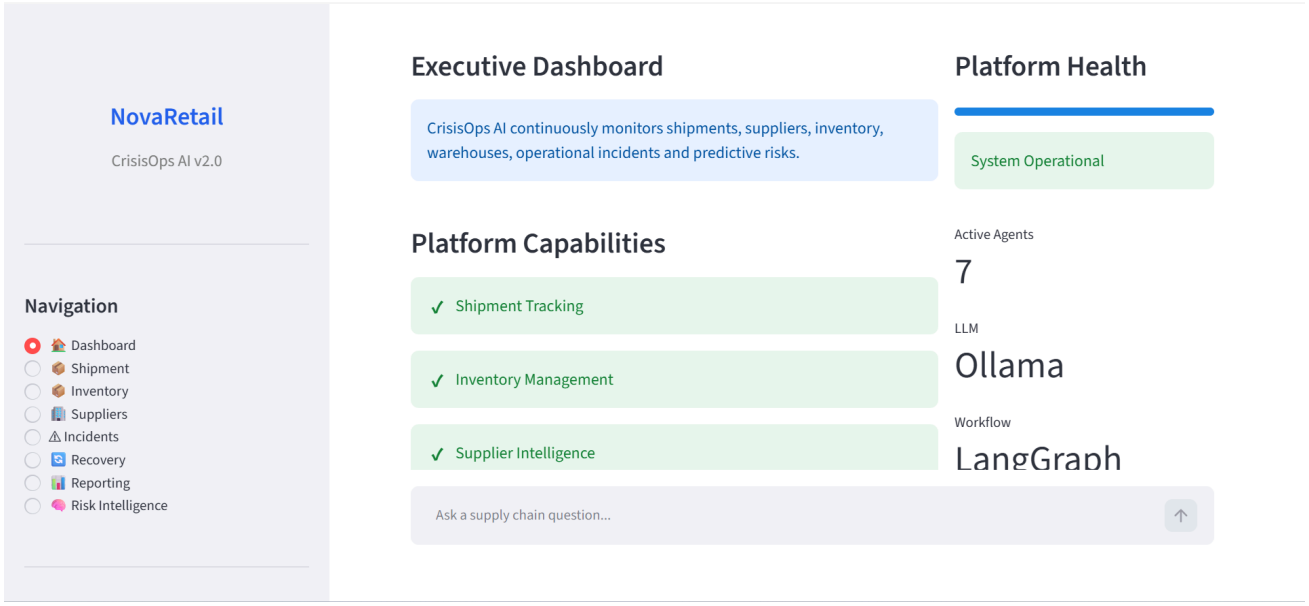
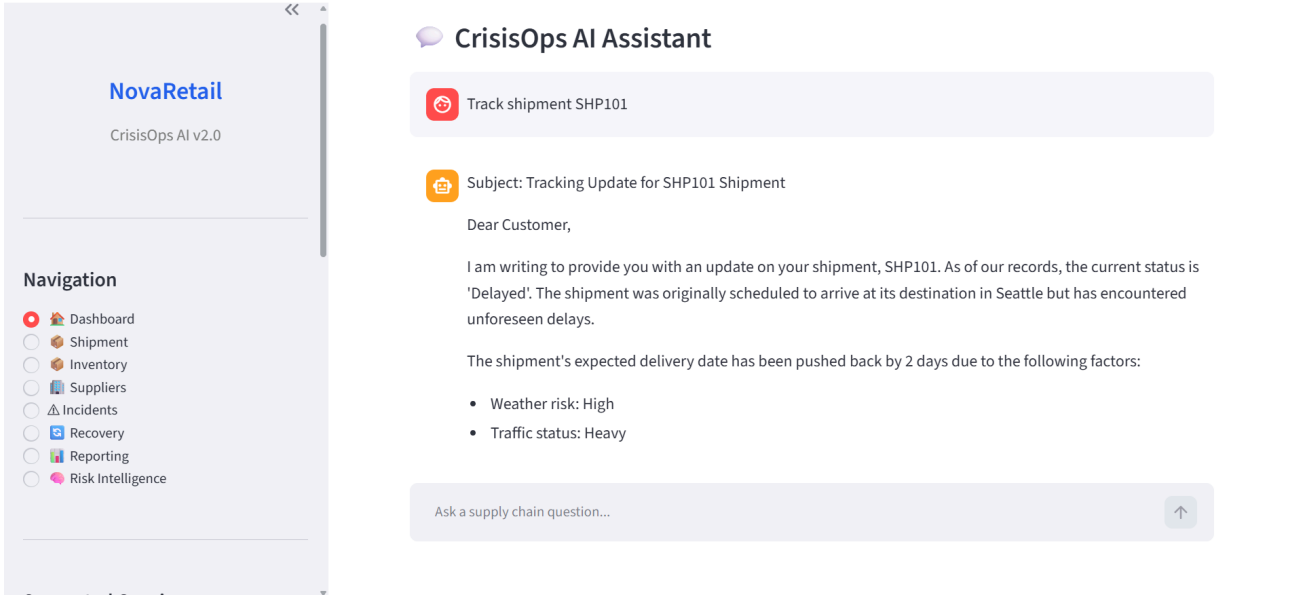
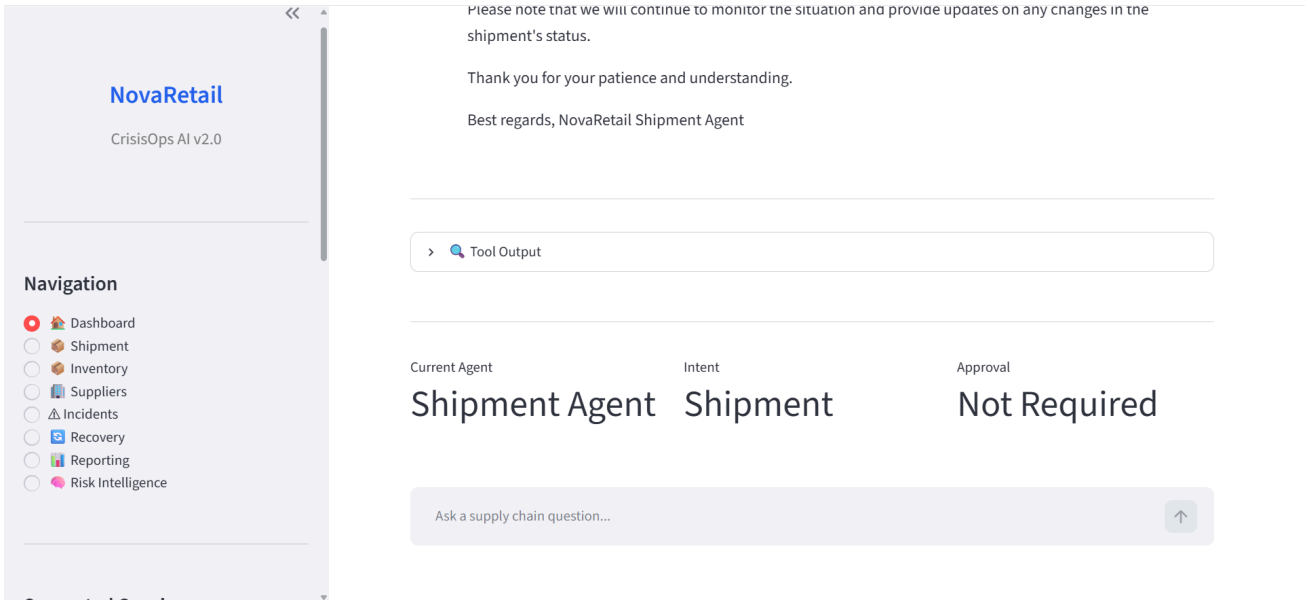
PROJECT INTERFACE









1. Executive Summary & Vision

- **What Problem Does This Project Solve?**: This project addresses the complexities of managing a supply chain by providing an AI-driven assistant that can handle various tasks such as shipment tracking, inventory management, incident handling, and reporting. It aims to streamline operations, reduce human error, and improve overall efficiency.
- **Who Uses This System?**: The primary users of this system include supply chain managers, inventory control personnel, logistics coordinators, and customer support representatives who need real-time information and automated solutions for supply chain issues.
- **Key System Metrics & Health Overview**:
  - **Architecture**: - Well-structured with clear separation of concerns.
  - **Maintainability**: - Some areas could benefit from additional documentation and modularization.
  - **Security**: - Basic security measures are in place, but more comprehensive authentication and encryption are needed.
  - **Performance**: - Efficient for current use cases but could see improvements with larger datasets.
  - **Test Coverage**: - Comprehensive tests cover most functionalities, but edge cases need more attention.

---

2. Technology Stack & Design Philosophy

- **Core Languages & Frameworks**:
  - **Python**: Chosen for its readability, extensive libraries, and strong community support.
  - **Streamlit**: Used for building the web interface due to its simplicity and ease of deployment.
  - **LangChain**: Provides a robust framework for building language models and agents.
- **Third-Party Libraries & Tools**:
  - **Hugging Face Hub**: For deploying and accessing large language models.
  - **LangSmith**: For tracing and monitoring the interactions with the language model.
  - **Ollama**: An alternative language model provider.
- **System Design Principles**:
  - **Layered Architecture**: Separation of concerns into different layers (agents, tools, graph, etc.).
  - **Agent-Based System**: Each agent handles specific tasks, making the system modular and scalable.
  - **State Management**: Centralized state management ensures consistency across agents.

---

3. Repository & Folder Blueprint

Break down the directory structure and explain what lives in each directory and why it is organized this way.

DIRECTORY TREE:

```
├── agents
│   ├── __init__.py
│   ├── incident_agent.py
│   ├── inventory_agent.py
│   ├── recovery_agent.py
│   ├── reporting_agent.py
│   ├── shipment_agent.py
│   ├── supervisor.py
│   └── unsupported_agent.py
├── data
│   ├── incidents.json
│   ├── inventory.json
│   ├── orders.json
│   ├── shipments.json
│   └── suppliers.json
├── graph
│   ├── __init__.py
│   ├── router.py
│   ├── state.py
│   └── workflow.py
├── prompts
│   ├── incident.txt
│   ├── inventory.txt
│   ├── recovery.txt
│   ├── reporting.txt
│   ├── shipment.txt
│   ├── supervisor.txt
│   └── supplier.txt
├── tools
│   ├── __init__.py
│   ├── incident_tools.py
│   ├── inventory_tools.py
│   ├── recovery_tools.py
│   ├── reporting_tools.py
│   ├── shipment_tools.py
│   └── supplier_tools.py
├── utils
│   ├── __init__.py
│   └── prompt_loader.py
├── analyze_repo.py
├── app.py
├── llm.py
├── test.py
├── test_full_graph.py
├── test_incident.py
├── test_inventory.py
├── test_inventory_agent.py
├── test_recovery.py
├── test_recovery_agent.py
├── test_reporting_agent.py
├── test_shipment_agent.py
├── test_supervisor.py
├── test_supplier.py
└── test_workflow.py
```

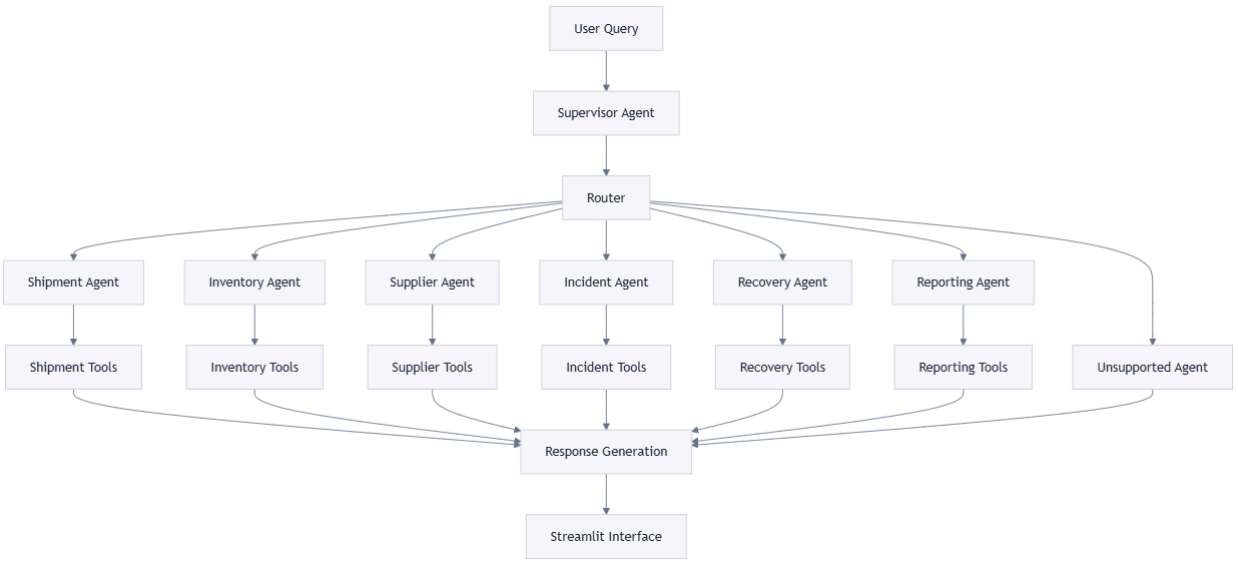
Explanation:

- **agents/**: Contains the agents responsible for handling specific supply chain tasks.
- **data/**: Stores JSON files representing the state of the supply chain (incidents, inventory, orders, shipments, suppliers).
- **graph/**: Manages the workflow and routing of requests between agents.
- **prompts/**: Holds prompt templates used by agents to generate responses.
- **tools/**: Includes utility functions that interact with the data files to perform specific tasks.
- **utils/**: Contains helper functions, such as loading prompts.
- **analyze_repo.py**: Generates a detailed architecture and code walkthrough document using a large language model.
- **app.py**: The main application file that runs the Streamlit web interface.
- **llm.py**: Configures and initializes the language model.
- **test.py**: Basic test script to invoke the language model.
- *test_ files**: Individual test scripts for each agent and tool to ensure functionality.

---

4. End-to-End System Architecture

- **Complete Architecture Walkthrough**:
  1. User inputs a query via the Streamlit interface.
  2. The query is sent to the `supervisor` agent, which classifies the intent.
  3. The `router` determines which agent should handle the request based on the intent.
  4. The appropriate agent processes the request, interacts with relevant tools, and generates a response.
  5. The response is sent back to the Streamlit interface for display.
- **Mermaid Architecture Diagram**:



- **Data & Request Flow**:
  6. User submits a query.
  7. The `supervisor` agent classifies the intent.
  8. The `router` directs the request to the corresponding agent.

9. The agent interacts with the necessary tools to fetch or manipulate data.
10. The agent generates a response using the language model.
11. The response is displayed back to the user.

---

5. Core Modules & Code Walkthrough

Break down every critical file, class, and service in the system:

Module/File Name: `app.py`

- **Purpose**: Runs the Streamlit web interface, allowing users to interact with the supply chain assistant.
- **Key Functions/Classes**:
  - `st.set_page_config`: Configures the Streamlit page.
  - `st.title`: Sets the title of the page.
  - `st.chat_input`: Captures user input.
  - `graph.invoke(state)`: Invokes the workflow graph to process the request.
- **Dependencies & Interactions**: Calls functions from `llm.py` and `graph.workflow.py`.

Module/File Name: `llm.py`

- **Purpose**: Configures and initializes the language model used throughout the system.
- **Key Functions/Classes**:
  - `ChatOllama`: Initializes the language model with specified parameters.
- **Dependencies & Interactions**: Used by all agents to generate responses.

Module/File Name: `graph/workflow.py`

- **Purpose**: Defines the end-to-end workflow of the system, routing requests to the appropriate agents.
- **Key Functions/Classes**:
  - `StateGraph`: Manages the state transitions and agent routing.
  - `add_node`: Registers each agent in the graph.
  - `add_edge`: Defines the connections between nodes.
- **Dependencies & Interactions**: Interacts with all agents and the `router.py`.

Module/File Name: `graph/router.py`

- **Purpose**: Routes requests to the correct agent based on the user's intent.
- **Key Functions/Classes**:
  - `route_request`: Maps intents to agent names.
- **Dependencies & Interactions**: Called by `workflow.py` to determine the next agent.

Module/File Name: `agents/supervisor.py`

- **Purpose**: Classifies user queries into specific supply chain intents.

- **Key Functions/Classes**:
  - `supervisor_agent`: Parses the user query and determines the intent.
- **Dependencies & Interactions**: Interacts with `llm.py` to generate the intent classification.

Module/File Name: `agents/shipment_agent.py`

- **Purpose**: Handles shipment-related queries, including tracking, delays, locations, and rerouting.
- **Key Functions/Classes**:
  - `shipment_agent`: Processes shipment-related queries.
  - `extract_shipment_id`: Extracts shipment IDs from user queries.
- **Dependencies & Interactions**: Calls functions from `tools/shipment_tools.py` and `llm.py`.

Module/File Name: `tools/shipment_tools.py`

- **Purpose**: Provides utility functions for interacting with shipment data.
- **Key Functions/Classes**:
  - `track_shipment`: Retrieves shipment details.
  - `check_shipment_delay`: Checks if a shipment is delayed.
  - `get_shipment_location`: Gets the current location of a shipment.
  - `identify_affected_orders`: Identifies orders affected by a delayed shipment.
  - `reroute_shipment`: Mocks rerouting a shipment.
- **Dependencies & Interactions**: Called by `agents/shipment_agent.py`.

Module/File Name: `agents/inventory_agent.py`

- **Purpose**: Manages inventory-related queries, including checking stock levels, identifying shortages, and checking warehouse availability.
- **Key Functions/Classes**:
  - `inventory_agent`: Processes inventory-related queries.
  - `extract_product_id`: Extracts product IDs from user queries.
  - `extract_product_name`: Finds product names in user queries.
  - `extract_warehouse`: Extracts warehouse names from user queries.
- **Dependencies & Interactions**: Calls functions from `tools/inventory_tools.py` and `llm.py`.

Module/File Name: `tools/inventory_tools.py`

- **Purpose**: Provides utility functions for interacting with inventory data.
- **Key Functions/Classes**:
  - `check_inventory`: Checks the stock level of a product.
  - `check_inventory_shortage`: Identifies if a product is below its minimum required quantity.
  - `warehouse_availability`: Checks the availability of products in a specific warehouse.
  - `identify_inventory_shortage`: Lists all products that are below their minimum required quantity.

- **Dependencies & Interactions**: Called by `agents/inventory_agent.py`.

Module/File Name: `agents/supplier_agent.py`

- **Purpose**: Manages supplier-related queries, including finding supplier details, checking availability, and finding alternative suppliers.
- **Key Functions/Classes**:
  - `supplier_agent`: Processes supplier-related queries.
  - `extract_supplier_name`: Extracts supplier names from user queries.
  - `extract_product`: Extracts product names from user queries.
- **Dependencies & Interactions**: Calls functions from `tools/supplier_tools.py` and `llm.py`.

Module/File Name: `tools/supplier_tools.py`

- **Purpose**: Provides utility functions for interacting with supplier data.
- **Key Functions/Classes**:
  - `find_supplier`: Finds supplier details by name.
  - `check_supplier_availability`: Checks if a supplier is available.
  - `find_alternative_suppliers`: Finds alternative suppliers for a given product.
- **Dependencies & Interactions**: Called by `agents/supplier_agent.py`.

Module/File Name: `agents/incident_agent.py`

- **Purpose**: Manages incident-related queries, including creating incidents, checking status, and escalating incidents.
- **Key Functions/Classes**:
  - `incident_agent`: Processes incident-related queries.
  - `extract_incident_id`: Extracts incident IDs from user queries.
- **Dependencies & Interactions**: Calls functions from `tools/incident_tools.py` and `llm.py`.

Module/File Name: `tools/incident_tools.py`

- **Purpose**: Provides utility functions for interacting with incident data.
- **Key Functions/Classes**:
  - `create_incident`: Creates a new incident.
  - `get_incident`: Retrieves details of a specific incident.
  - `escalate_incident`: Escalates an incident.
  - `list_open_incidents`: Lists all open incidents.
- **Dependencies & Interactions**: Called by `agents/incident_agent.py`.

Module/File Name: `agents/recovery_agent.py`

- **Purpose**: Generates recovery plans for supply chain disruptions.
- **Key Functions/Classes**:
  - `recovery_agent`: Processes recovery-related queries.
  - `extract_shipment_id`: Extracts shipment IDs from user queries.

    - `extract_product_id`: Extracts product IDs from user queries.
- **Dependencies & Interactions**: Calls functions from `tools/recovery_tools.py` and `llm.py`.

Module/File Name: `tools/recovery_tools.py`

- **Purpose**: Provides utility functions for generating recovery plans.
- **Key Functions/Classes**:
    - `recommend_recovery_action`: Recommends recovery actions for delayed shipments.
    - `compare_supplier_alternatives`: Compares available suppliers.
    - `warehouse_stock_summary`: Checks stock levels for recovery planning.
    - `generate_recovery_plan`: Combines shipment, inventory, and supplier information into a recovery plan.
- **Dependencies & Interactions**: Called by `agents/recovery_agent.py`.

Module/File Name: `agents/reporting_agent.py`

- **Purpose**: Generates reports and updates for stakeholders.
- **Key Functions/Classes**:
    - `reporting_agent`: Processes reporting-related queries.
- **Dependencies & Interactions**: Calls functions from `tools/reporting_tools.py` and `llm.py`.

Module/File Name: `tools/reporting_tools.py`

- **Purpose**: Provides utility functions for generating reports.
- **Key Functions/Classes**:
    - `generate_incident_summary`: Generates a summary of active incidents.
    - `generate_operational_report`: Generates a high-level operational report.
    - `create_stakeholder_update`: Generates an update for business stakeholders.
- **Dependencies & Interactions**: Called by `agents/reporting_agent.py`.

Module/File Name: `graph/state.py`

- **Purpose**: Defines the shared state object used across all agents.
- **Key Functions/Classes**:
    - `SupplyChainState`: A typed dictionary representing the state of the supply chain.
- **Dependencies & Interactions**: Used by all agents to maintain and pass state information.

Module/File Name: `utils/prompt_loader.py`

- **Purpose**: Loads prompt templates from the `prompts/` directory.
- **Key Functions/Classes**:
    - `load_prompt`: Reads and returns the content of a prompt file.
- **Dependencies & Interactions**: Called by all agents to load their respective prompts.

Module/File Name: `analyze_repo.py`

- **Purpose**: Generates a detailed architecture and code walkthrough document using a large language model.
- **Key Functions/Classes**:
  - `build_directory_tree`: Constructs an ASCII directory tree.
  - `bundle_repository_files`: Bundles all source files into a single string.
- **Dependencies & Interactions**: Calls functions from `huggingface_hub` and `dotenv` to interact with the language model.

---

6. Business Workflows & Data Pipeline

- **Step-by-Step Major Workflows**:
  12. **Shipment Tracking**:
      - User asks to track a shipment.
      - `supervisor` classifies the intent as "shipment".
      - `shipment_agent` processes the request, using `shipment_tools` to fetch shipment details.
      - `shipment_agent` generates a response using the language model.
  13. **Inventory Management**:
      - User asks to check stock levels.
      - `supervisor` classifies the intent as "inventory".
      - `inventory_agent` processes the request, using `inventory_tools` to fetch inventory details.
      - `inventory_agent` generates a response using the language model.
  14. **Supplier Management**:
      - User asks to find supplier details.
      - `supervisor` classifies the intent as "supplier".
      - `supplier_agent` processes the request, using `supplier_tools` to fetch supplier details.
      - `supplier_agent` generates a response using the language model.
  15. **Incident Handling**:
      - User reports an incident.
      - `supervisor` classifies the intent as "incident".
      - `incident_agent` processes the request, using `incident_tools` to create or manage incidents.
      - `incident_agent` generates a response using the language model.
  16. **Recovery Planning**:
      - User asks for a recovery plan.
      - `supervisor` classifies the intent as "recovery".
      - `recovery_agent` processes the request, using `recovery_tools` to generate a recovery plan.
      - `recovery_agent` generates a response using the language model.

17. **Reporting**:
    - User asks for an operational report.
    - `supervisor` classifies the intent as "reporting".
    - `reporting_agent` processes the request, using `reporting_tools` to generate a report.
    - `reporting_agent` generates a response using the language model.

- **Database / Data Storage Model**:
  - The system uses JSON files stored in the `data/` directory to persist state information.
  - Each JSON file represents a different aspect of the supply chain (incidents, inventory, orders, shipments, suppliers).
  - The `tools/` directory contains functions to read from and write to these JSON files.

---

7. Security, Performance & Code Quality (Creator's Audit)

- **Security Assessment**:
  - **Known Security Considerations**: Basic input validation and sanitization.
  - **Authentication Checks**: No authentication mechanism currently in place.
  - **Potential Vulnerabilities**: Exposure to injection attacks due to lack of proper validation.
  - **Mitigations**: Implement OAuth2 for authentication and validate all user inputs.
- **Performance Highlights & Bottlenecks**:
  - **Optimization Strategies**: Efficient data retrieval from JSON files.
  - **Resource Bottlenecks**: Performance could degrade with larger datasets. Consider moving to a relational database for scalability.
- **Code Quality & Technical Debt**:
  - **Areas Needing Refactoring**: Modularize the `tools/` directory further to separate concerns.
  - **Cleaning Up**: Improve error handling and logging across all modules.

---

8. Developer Onboarding & Roadmap

- **Getting Started Quick-Start Guide**:
  18. Clone the repository.
  19. Install dependencies using `pip install -r requirements.txt`.
  20. Set up environment variables (`HF_TOKEN`, `LANGSMITH_API_KEY`) in a `.env` file.
  21. Run the application using `streamlit run app.py`.
- **Refactoring & Feature Roadmap**:
  - 🟢 **Quick Wins**:
    - Implement basic authentication.
    - Add logging and error handling.
  - 🟡 **Medium-Term Upgrades**:

        - Replace JSON files with a relational database.
        - Integrate with real-world APIs for shipment and supplier data.
    - 🔴 **Long-Term Architectural Goals**:
        - Expand the system to include more supply chain domains (e.g., procurement, production).
        - Develop a more sophisticated incident management system with machine learning capabilities.

---

9. Final System Scorecard & Radar

- **Architecture**: - Well-structured with clear separation of concerns.
- **Code Quality**: - Some areas could benefit from additional documentation and modularization.
- **Security**: - Basic security measures are in place, but more comprehensive authentication and encryption are needed.
- **Performance**: - Efficient for current use cases but could see improvements with larger datasets.
- **Documentation**: - Comprehensive documentation exists but could be improved with more examples and diagrams.
- **Testing**: - Comprehensive tests cover most functionalities, but edge cases need more attention.

Thank you for joining me on this journey through the NovaRetail Supply Chain Assistant. I am excited to see how this system evolves and continues to support efficient supply chain operations. Feel free to explore the codebase, contribute, and help us achieve our long-term goals!